Rambus

Security IP
EIP-130

Root of Trust engine
Driver Development Kit v3.1

User Guide

Document Revision: A
Document Date: 2022-02-09
Document Number: 007-130310-308
Document Status: Accepted

© Rambus Inc. • rambus.com CONFIDENTIAL

Copyright 2015-2022 Rambus Inc. This document contains information which is proprietary and confidential, and which is protected under patents, copyrights, and/or other IP rights of Rambus Inc. If you are not the intended recipient of this material, please destroy this document and inform Rambus at +1 408 463 8000 or sipsupport@rambus.com immediately.

Rambus Inc. Corporate Headquarters
4453 North First Street, Suite 100
San Jose, CA 95134
Phone: +1 408-462-8000
Website : https://www.rambus.com/
Contact : sipsupport@rambus.com

Rambus ROTW Holding B.V.
Boxtelseweg 26A
5261 NE Vught
The Netherlands
Phone: +31-73-6581900

Table of Contents

Table of Contents 3
List of Tables 4
List of Figures 4
Document Revision History 5
1 Introduction 6
1.1 Purpose 6
1.2 Related Documents 6
1.3 Target Audience 6
1.4 Abbreviations, Acronyms and Definitions 6
2 DDK Concept 8
2.1 Goal 8
2.2 Terminology 8
2.3 Contents 9
2.4 Building and Running DDK Examples 9
3 Typical Driver Development Problems 12
4 DDK Problem-Solution Map 15
5 DDK Installation 16

List of Tables

Table 1 Typical driver development problems 15
Table 2 *DDK* folders after installation 16

List of Figures

Figure 1 *EIP-130* Driver Kernel-Space and User-Space Deployments 13
Figure 2 MemXS and Driver Framework User-Space deployment 14
Figure 3 Multiple application support example 14

Document Revision History

Doc Rev	Page(s) Section(s)	Date	Author	Purpose of Revision
A	All	2022-02-09	MHO	• First release for v3.1, based on v3.0 RevC.

1 Introduction

1.1 Purpose

The *EIP-130 Root of Trust engine* (also referred to as the *Root of Trust / RT or VaultIP*) is a product that protects valuable assets (Security Critical Parameters) such as secret keys for security sensitive applications. Additionally, the *EIP-130* embeds hardware implementations of cryptographic algorithms supporting a variety of operations and modes.

The *Driver Development Kit (DDK)* provides a set of solutions required to develop drivers for the *EIP-130*. Solutions for typical driver development problems are provided in various forms such as documentation, API specification and example code.

Note: The DDK contains Example Drivers for the *EIP-130* hardware for the ARM or x86 based Linux platforms. However, the main purpose of this toolkit is to provide support for the customized driver development for HW platforms and OS's chosen by the DDK users.

The purpose of this document is to provide:

- High level overview of *DDK* components.
- Description of typical driver development problems and solutions provided in the *DDK.*
- References to APIs that can be used in customized drivers.
- References to example code that can be used as a base for customized drivers.
- Reference to the other documents useful for the understanding of driver development.

Note: Please see the "Dependencies and Compatibility" section of the corresponding release notes for hardware version specific compatibility information.

1.2 Related Documents

Ref	Document	Document Number
[1]	Security IP Driver Framework Porting Guide	007-900450-304
[2]	EIP-130 Driver User Guide	007-130310-301
[3]	EIP-130 DDK Test Specification	007-130310-310
[4]	EIP-130 Firmware Reference Manual	007-130xx0-204

This information is correct at the time of document release. Rambus reserves the right to update the related documents without updating this document. Please contact Rambus for the latest document revisions.

The preferred method for getting technical support is to use our online support system at https://sipsupport.rambus.com. If you do not have an account yet for this system, please contact Rambus technical support (sipsupport@rambus.com).

1.3 Target Audience

This document is for software developers who want to create customized drivers for the *EIP-130* hardware.

1.4 Abbreviations, Acronyms and Definitions

Acronym	Description
3DES	Triple-DES
AES	Advanced Encryption Standard
API	Application Program Interface
ARIA	South-Koran cipher algorithm
BSP	Board Support Package
CBC	Cipher Block Chaining

CCM	Counter with CBC-MAC
ChaCha20	Stream cipher created by Daniel J. Bernstein
CMAC	Cipher MAC
CTR	CounTeR
DDK	Driver Development Kit
DES	Data Encryption Standard
DMA	Direct memory Access
DRBG	Deterministic Random Bit Generator
ECB	Electronic CodeBook
ECC	Elliptic Curve Cryptography
ECDSA	Elliptic Curve Digital Signature Algorithm
EdDSA	Edwards-curve Digital Signature Algorithm
EIP	Embedded Intellectual Property
FW	Firmware
GC	Global Control
GCM	Galois/Counter Mode
HMAC	Hash MAC
HUK	Hardware Unique Key (Root Key)
HW	Hardware
IC	Interrupt Coalescing
ICM	Integer Counter Mode
KAT	Known-Answer Test
MAC	Message Authentication Code
Milenage	3GPP authentication and key generation functions
NOP	No Operation
NRBG	Non-deterministic Random Bit Generator
OS	Operating System
OTP	One-Time Programmable
PKCS	Public-Key Cryptography Standards
PKI	Public Key Infrastructure
Poly1305	Cryptographic message authentication code created by Daniel J. Bernstein
PSS	Probabilistic Signature Scheme
RSA	Rivest, Shamir, Adleman
SCP	Security Critical Parameter
RoT	Root of Trust
SIV	Synthetic Initialization Vector
SM2	Chinese asymmetric algorithm
SM3	Chinese symmetric hash algorithm
SM4	Chinese symmetric cipher algorithm
SW	Software
TRNG	True Random Number Generator
UMDevXS	User Mode Device access (XS)
VAL	VaultIP Abstraction Layer (the API for applications)
VEX	VaultIP EXchange
XEX	XOR Encrypt XOR
XOR	eXclusive OR
XTS	XEX Tweakable Block Cipher with Ciphertext Stealing

2 DDK Concept

2.1 Goal

The goal of the *DDK* is to reduce the development effort of customized drivers (C-Drivers) for the *EIP-130* hardware and enable faster integration into the customer's system applications. Example Drivers for the *EIP-130* hardware are provided that can be used as a starting point for development of a customized driver.

Note: The provided Example Drivers are tested only with the HW platform and OS indicated in the test report. The DDK can be used to develop a driver for other HW platforms and OSs.

2.2 Terminology

The table below describes various terms used throughout the document.

Term	Description
Adapter	Software component that needs to be customized for specific hardware platform, OS, use cases, system application and required driver API.
Global Adapter	Adapter used in *EIP-130* Example Combined Driver and Global Control Driver intended for *EIP-130* hardware initialization/setup.
Kernel-space deployment	Driver where the API is deployed in the application Kernel-Space memory domain, e.g. the application with the driver runs in the kernel-space.
User-space deployment	Driver where the API is deployed in the application User-Space memory domain, e.g. the application with the driver runs in the user-space.
Driver Framework	Collection of software interface (API) specifications typically used by EIP Driver Libraries and other driver components. This framework defines how to use EIP Driver Libraries in a driver.
EIP Driver Library	Software library that contains control functionality for a given EIP hardware block only and is free of any other hardware or software functionality not related to the EIP hardware block.
EIP-130 Kernel Support Driver	An Example Driver that implements the support functionality for the *EIP-130* driver(s) deployed in Linux user-space. It also implements the support functionality for the shared device access for drivers deployed in Linux User-Space as well as in the Linux kernel-space.
EIP-130 Combined Driver	An Example Driver that implements the Global Control API, VAL API and VEX API. It may be used to verify the *EIP-130* Driver Library including Global Control, VAL API and other *DDK* software components. Some customization effort may be required from users to start using the driver in their system application.
EIP-130 Global Control Driver	Example Driver that implements the Global Control API. This API allows to initialize the *EIP-130* and to get the engine status information. Only one instance of this API is supported. It may be used to verify the *EIP-130* Driver Library including only the Global Control functionality in one driver and other *DDK* software components. Some customization effort may be required from users to start using the driver in their system application.
EIP-130 VAL Driver	An Example Driver that implements the VAL API and VEX API. This Driver may be used to verify the *EIP-130* Driver Library including other *DDK* software components. This driver must be used together with the *EIP-130* Global Control Driver. Some customization effort may be required from users to start using the driver in their system application.
VAL Test Tool	Test Application used to verify most of the VAL API functionality that is provided by the Driver with the integrated *EIP-130* Driver Library. Note that this Test Application can also be used as example how to use the VAL API.

Term	Description
DemoApp	A Demo Application provides a reference implementation of the VAL API use. This DDK provides the following Demo Applications: • DemoApp_VAL Use example for common VAL API functions. • DemoApp_VAL_Specials Use example for special VAL API functions. • DemoApp_SecureDebug Use example for Authentication Unlock and Secure Debug. • DemoApp_OTPProgram Use example for writing an item to OTP. • DemoApp_OTPKeyBlob Use example for generating OTP key blob with *EIP-130*..
Customized Driver (C-Driver)	*EIP-130* driver customized for specific use cases in a system application. Usually requires significant development effort. After the customization, the driver is ready to use in a customer's system application.

2.3 Contents

The *DDK* provides a set of re-usable software components used to solve typical problems encountered by C-Driver development. The *DDK* includes re-usable software components, EIP Driver Libraries that allows full utilization of all EIP hardware features, along with extensive documentation for the provided SW components and EIP Driver Libraries.

Note: The DDK is designed to reduce the required driver development effort but cannot eliminate it.

Solutions are provided in the *DDK* in the form of:

- Re-usable driver architecture and design principles (see Driver Framework Porting Guide [1]).
- Re-usable APIs (see Driver Framework Porting Guide [1] and C header-files).
- Documentation with guidelines on how to implement the provided APIs in a customer-specific way, e.g. guidelines for the customization effort (Porting guidelines and implementation notes).
- Re-usable SW components (C source code, user documentation):
  - *EIP-130* Driver Library.
  - *Security IP-201* Driver Library.
  - VAL API
  - VEX API
  - VAL Test Tool.
  - Re-usable software modules and libraries.
- Example Drivers for demonstrating how to use various DDK software components for making customized drivers specific to the *EIP-130* hardware.

The *EIP-130* and *Security IP-201* Driver Library are the most important software components included in the *DDK*. The *DDK* targets driver development for this EIP HW block integrated in a chip. The *DDK* does not include development tools such as C compiler, linker, debugger, profiler, or IDE.

2.4 Building and Running DDK Examples

Every *DDK* example and test tool component (see Example and TestTools folders in Table 2) includes a respective User Guide or a readme file that explains how to use it. See the respective 'docs' directory for each example. The Demo Applications and Test Tools dependent on the Modules.symvers file from the Driver build, and therefore require that the driver be built first.

The Build tools must be available in the path environment variable.

Most of the scripts and make files depend on environment variables that must be set up:

- `ARCH`: selects the target architecture, the *DDK* supports "x86", "arm" and "arm64".
- `HOST_HW_PLATFORM`: the hardware platform of the host that runs the driver, the DDK supports "FPGA_Virtex6_PCI", "FPGA_Zynq_ZC702" (also applies to Zynq ZC706) or "FPGA_V2M_Juno" and it is possible to define additional customized values.
- `DDK_HOST64`: set to "y" if the host is a 64-bit platform.

- DDK_DEVICE64: always “y” for this DDK.
- KERNEL_DIR: path to the Linux kernel source tree.
- CROSS_COMPILE: cross-compile tool prefix.
- CROSS_LIB_PATH: path to C libraries used with cross compiler.
- USER_CFLAGS: special user-specific compiler flags that must be used in the build operations.
- USER_LFLAGS: special user-specific linker flags that must be used in the build operations.
- EARLY_USER_LFLAGS: special user-specific linker flags that must be used in the build operations, passed to the linker before the standard C runtime library and USER_LFLAGS flags.
- BUILD_CONFIGS: defines specific configurations that must be built and regressed, require for regression only.
- DDKDIR: path to DDK, require for regression only.

MicroBlaze specific environment variables:

- XSDK_PATH: Xilinx SDK location, contains the GNU tool chain, headers, and libraries.
- XBSP_PATH: Xilinx MicroBlaze FreeRTOS Board Support Package (BSP), contains the Xilinx Zynq ZC706 FPGA bit stream, first stage boot loader, MicroBlaze and FreeRTOS headers, and libraries.
- REGRESS_LIB: list of all the DDK demo applications and test tool that should be linked with the DDK regression shell as static libraries.
- REGRESS_LIB_PATH: location of the DDK demo applications and test tool libraries.
- PATH: path to various binaries required by the build system

The DDK installation script will write the required environment variables into the setvars.sh file that can be sourced to set them.

Example for 64-bit x86 with Xilinx Virtex6 PCIe FPGA:

```
# export ARCH=x86
# export HOST_HW_PLATFORM=FPGA_Virtex6_PCI
# export DDK_HOST64=y
# export DDK_DEVICE64=y
# export KERNEL_DIR=/usr/src/linux
# export USER_LFLAGS=
# export EARLY_USER_LFLAGS=""
# export CROSS_COMPILE=
# export CROSS_LIB_PATH=
# export BUILD_CONFIGS=
# export DDKDIR=/home/dev/sandbox/DDK-130_vX.Y
# export WARNING_FLAGS=
```

Example for 32-bit ARM with Xilinx Zynq ZC706 FPGA:

```
# export ARCH=arm
# export HOST_HW_PLATFORM=FPGA_Zynq_ZC702
# export DDK_HOST64=n
# export DDK_DEVICE64=y
# export KERNEL_DIR=/opt/linux-zynq-zc706
# export USER_LFLAGS=
# export EARLY_USER_LFLAGS=""
# export CROSS_COMPILE=arm-linux-gnueabihf-
# export CROSS_LIB_PATH=/opt/Xilinx/SDK/gnu/aarch32/lin/gcc-arm-linux-gnueabi/
lib/gcc/arm-linux-gnueabihf
# export BUILD_CONFIGS=
# export DDKDIR=/home/dev/sandbox/DDK-130_vX.Y
# export WARNING_FLAGS=
```

Example for 64-bit ARM with ARM Juno FPGA:

```
# export ARCH=arm64
# export HOST_HW_PLATFORM=FPGA_V2M_Juno
# export DDK_HOST64=y
# export DDK_DEVICE64=y
# export KERNEL_DIR=/opt/linux-juno
# export USER_LFLAGS=
# export EARLY_USER_LFLAGS="-L/home/dev/sandbox/DDK-130_vX.Y/Examples/Driver/
build -lmemrepl -fno-builtin-memset -fno-builtin-memcpy"
# export CROSS_COMPILE=aarch64-linux-gnu-
# export CROSS_LIB_PATH=/opt/Android/toolchains/gcc-linaro-aarch64
# export BUILD_CONFIGS=
# export DDKDIR=/home/dev/sandbox/DDK-130_vX.Y
# export WARNING_FLAGS=
```

In this 64-bit ARM example the `libmemrepl.a` library implements and replaces the standard C runtime library `memset()` and `memcpy()` functions implementation.

Example for Little Endian MicroBlaze with Xilinx Zynq ZC706 FPGA:

```
# export ARCH=mb
# export HOST_HW_PLATFORM=FPGA_Zynq_ZC702
# export DDK_HOST64=n
# export DDK_DEVICE64=y
# export XSDK_PATH=/opt/Xilinx/SDK
# export XBSP_PATH=/opt/zynq-zc706-bsp
# export REGRESS_LIB_PATH="-L$DDKDIR/Examples/DemoApp_VAL/build -L$DDKDIR/
TestTools/TestTool_VAL/build"
# export REGRESS_LIB="-lda_val_combined_u -ltest_val_combined_u"
# export USER_CFLAGS="-mlittle-endian -fmessage-length=0 -mcpu=v10.0 -mxl-soft-
mul -Wl,--no-relax -ffunction-sections -fdata-sections -I$XBSP_PATH/include"
# export USER_LFLAGS="-mlittle-endian -Wl,--start-group,-lmbfreertos,-lxil,
-lfreertos,-lgcc,-lc,--end-group -mcpu=v10.0 -mxl-soft-mul -Wl,--no-relax -Wl,
--gc-sections"
# export PATH= $XSDK_PATH/gnu/microblaze/lin/bin:$XSDK_PATH/gnu/bin:$XSDK_PATH/
bin
# export EARLY_USER_LFLAGS=""
# export CROSS_COMPILE=mb-
# export CROSS_LIB_PATH=$XBSP_PATH/lib
# export BUILD_CONFIGS=
# export DDKDIR=/home/dev/sandbox/DDK-130_vX.Y
# export WARNING_FLAGS=
```

In this MicroBlaze example, the `libxil.a`, `libfreertos.a`, `libgcc.a` and `libc.a` libraries come from the folder `$CROSS_LIB_PATH`. The Xilinx MicroBlaze and FreeRTOS headers come from the folder `$XBSP_PATH/include`. The `libmbfreertos.a` library comes from the folder `$DDKDIR/TestTools/Regress_vip/mb_freertos`, which is also the build folder for the MicroBlaze regression system shell. The regression system shell includes two applications, the Demo App VAL and the Test Tool VAL, built and linked as static libraries `libda_val_combined_u.a` and `libtest_val_combined_u.a` respectively.

3 Typical Driver Development Problems

This section lists typical problems that a device driver developer must address. Some of these problems may not be applicable because they depend on the type of the device.

- Map driver API onto HW block SW interface so that high level SW application can utilize the functionality implemented in the EIP HW block.
- Compatibility with native OS Driver Model and OS API so the driver can be used with that OS.
- Concurrent context synchronization (CCS), e.g. synchronizing access to resources (including EIP HW block resources) shared between concurrent execution contexts such as threads and interrupts.
- Concurrent device synchronization (CDS), e.g. synchronizing access to resources shared between EIP HW block and the host CPU.
- EIP HW block control functionality for data I/O that includes all required operations on the EIP HW block before it can start to input and output data, as well as operations related to data input and output.
- Provide access to static resources such EIP HW 'memory mapped' I/O registers and internal memories shared between host CPU and EIP HW block:
  - Resolve endianness differences between the host CPU and the EIP HW block.
  - Read and write static device resources via the appropriate hardware paths.
  - Map static device resources into memory domain of the application requiring the EIP services.
- Provide access to dynamic resources such as DMA buffers shared between host CPU and EIP HW block:
  - Dynamic resource (buffer) allocation, buffer address and size alignment.
  - Dynamic resource (buffer) address-translation when a system uses several types of addresses such as virtual, logical, physical.
  - Data-cache coherency management, when required.
- Interrupt dispatching and servicing across heterogeneous memory domains (from kernel-space to user-space).
- Compatibility with C compiler, its C Run-Time Library and basic type definitions.

The *DDK* provides a set of solutions for these problems in various forms as well as guidelines for integrating these solutions in a device driver. Although the *DDK* attempts to solve these problems, driver development remains a creative and challenging task requiring solid experience and skills in the field.

Other considerations:

The *EIP-130* hardware contains a DMA engine. All the EIP HW blocks are Little Endian devices.

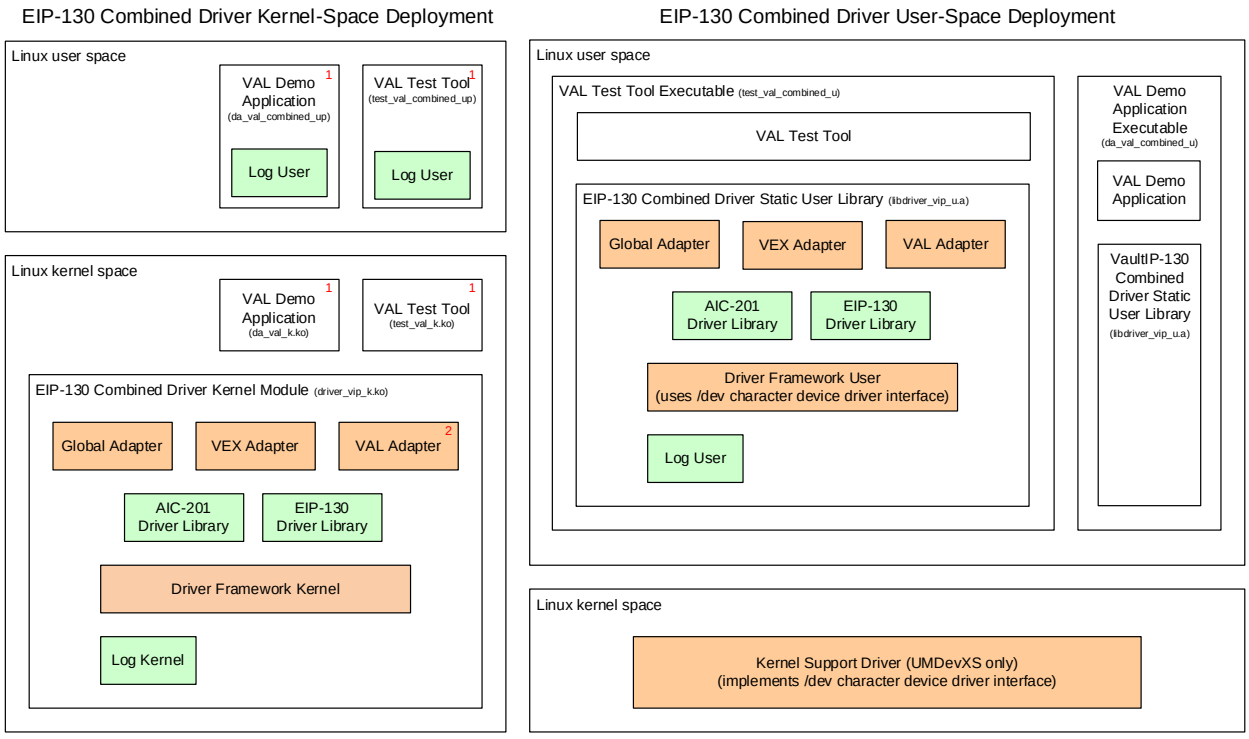
During the *DDK* verification

- Different hosts and targets were used for verification: Little Endian x86(-64) PC, Little Endian ARM Cortex-A9 and Big Endian MicroBlaze on the Xilinx Zynq-7000 SoC (ZC706) board and Little Endian ARM64 Cortex A57/A53 on the ARM Juno board.
- The Linux OS with SLUB/SLAB as buffer allocator was used for DMA buffer allocation on both hosts.
- Different addresses were used to access the DMA buffers from the device (using DMA) and from the host (from Linux kernel space as well as from user space), requiring proper address translation (kernel-virtual and user-virtual to PCIe or AXI bus).
- The used x86 platform ensures the data cache coherency between the CPU and IP HW in hardware. The ARM and MicroBlaze platforms ensure the data cache coherency in (IP HW driver) software.

The software block diagrams below show how various *DDK* components are used in case of the Kernel-Space and User-Space *EIP-130* driver deployment. The following notation is used in these diagrams:



Figure 1 shows the Kernel-Space and User-Space deployments. Note that for the Kernel-Space deployment, the VAL Demo Application and VAL Test Tool are also shown in the user-space. In this case, the User-Space applications use the VAL API in combination with a proxy driver that is not shown. The VAL Test Tool can be used to verify the VAL API. Thus, this Test tool can be deployed in the Kernel-Space as well as in the user-space. The same is true for the Demo Application.



1 Note that only one of these applications (VAL Demo or VAL test tool in user space or kernel space) can be executed at a time.

2 The VAL API is directly accessible for kernel space applications. However, for user space applications it is proxied via VEX Proxy/ Stub, which is not shown.

Figure 1 *EIP-130* Driver Kernel-Space and User-Space Deployments

There can be only one Combined Driver deployment active at a time, e.g. either Kernel-Space driver or User-Space driver. In case the Combined Driver is deployed as Kernel-Space driver, a user-mode application can use the Kernel-Space driver via a proxy. The Combined Driver cannot be used concurrently with Global Control Driver or VAL Driver. The Combined Driver must be used together with the Kernel Support Driver (only UMDevXS functionality) for the User-Space deployment and without it for the Kernel-Space deployment. The Combined Driver can support only one application at a time, e.g. either the VAL Test Tool with the built-in *EIP-130* Example Combined Driver (User-Space driver deployment) or the MemXS Tool.

The MemXS Tool does not use the entire *EIP-130* Example Driver but only the Driver Framework implementation for the user space. Figure 2 shows the MemXS and Driver Framework deployment.

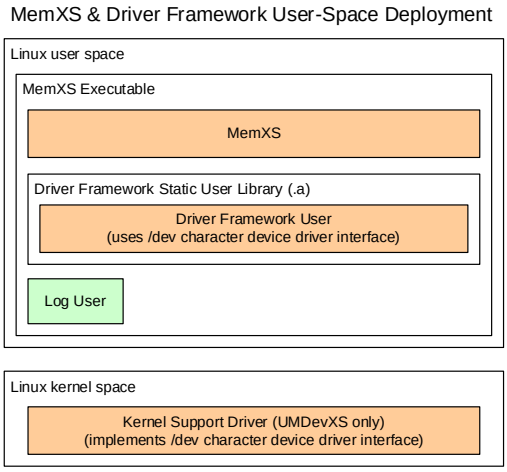
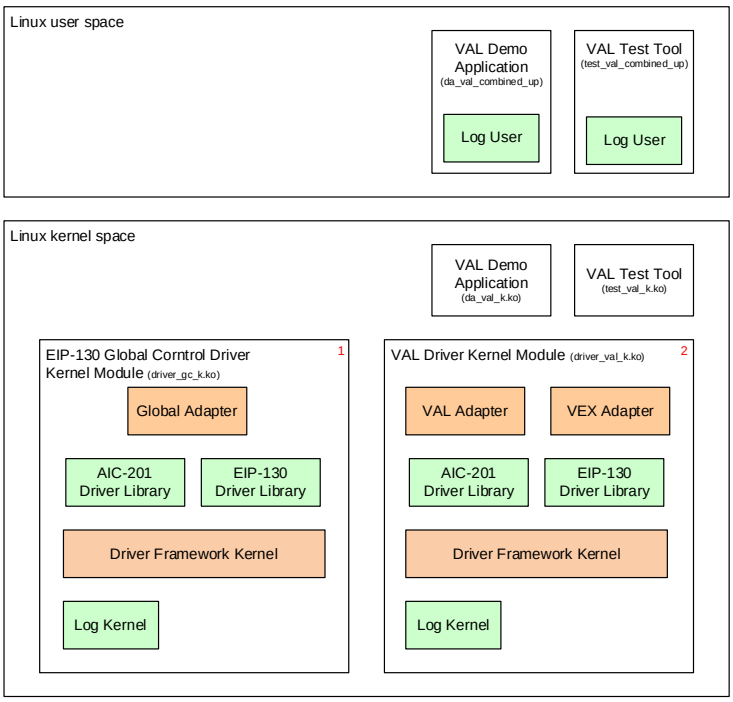


Figure 2 MemXS and Driver Framework User-Space deployment

This *DDK* provides also a solution for multiple concurrent applications support, because the *EIP-130* hardware can be accessed by only one application at a time. In that case, the VAL Driver (driver_val_k.ko) will control/manage the *EIP-130* hardware access. The block diagram below shows how the *EIP-130* Drivers can be deployed to support multiple concurrent applications, note that the VAL Test Tool and VAL Demo application are used as example.



1 One instance system wide.
2 One instance per Host.

Figure 3 Multiple application support example

Note that the User-Space applications use the VAL API in combination with a proxy driver that is not shown in Figure 3.

4 DDK Problem-Solution Map

The table below provides an overview of the typical driver development problems described in section 3 with corresponding solutions and *DDK* deliverables.

Table 1 Typical driver development problems

Typical driver development problem	DDK Solution	DDK Deliverables
Map driver API to HW block SW interface	• VAL API	• Driver API Specification as C header-files • Driver API Implementation Notes • Driver API (VAL API) • VAL Test Tool with test cases and test vectors for the VAL API
Compatibility with OS Driver Model and OS API	• Adapter • Driver Framework	• VAL and VEX Adapter C source code • Driver Framework Device & DMA Resource API implementation C source code
Concurrent context synchronization (CCS)	• Adapter • EIP-Driver Library User Guides	• VAL and VEX Adapter C source code • EIP-130 Driver Libraries C source code • Security IP-201 Driver Libraries C source code
Concurrent device synchronization (CDS)	EIP Driver Library	• EIP-130 Driver Libraries C source code • Security IP-201 Driver Libraries C source code
EIP HW block control functionality for data I/O	EIP Driver Library	• EIP-130 Driver Libraries C source code • Security IP-201 Driver Libraries C source code
Big Endian host vs. Little Endian HW block	Driver Framework: • Device API • DMA Resource API	• Driver Framework Porting guidelines • Driver Framework Device & DMA Resource API implementation in C source code
Read and write static device resources	Driver Framework: • Device API	• Driver Framework Porting guidelines • Driver Framework: Device API implementation C source code
Dynamic resource allocation, address and size alignment	• DMA Resource API	• Driver Framework Porting Guide • Driver Framework: DMA Resource API implementation C source code
Address translation for dynamic resource	• DMA Resource API • Adapter	• Driver Framework Porting Guide • Driver Framework: DMA Resource API implementation C source code • VEX Adapter C source code
Data cache coherency control	Driver Framework: • DMA Resource API (PreDMA / PostDMA)	• Driver Framework Porting Guide • Driver Framework: DMA Resource API implementation C source code
Compatibility with C-compiler, its Run-Time Library and basic types	Driver Framework: • C Run-Time API • Basic Types API	• Driver Framework Porting Guide • Driver Framework C Run-Time API C header files and implementation C source code • Driver Framework Basic Types API C header files and implementation C source code
Static device resources mapping into User-Space memory domain	UMDevXS	• Kernel support driver and its User-Space proxy implementation C source code

To find these *DDK* deliverables in the provided *DDK* package after the installation, refer to section 5.

The Adapter SW components as well as the Driver Framework API's are implemented for the example driver API's (VAL API and VEX API), hardware platform (x86, ARM and ARM64) and OS (Linux) used in the *DDK* verification environment.

The same applies for the UMDevXS component.

The EIP Driver Libraries are by design independent from the host HW and OS.

5 DDK Installation

Unpack the *DDK* archive then run the `install_ddk.sh` script once.

```
# tar -zxf 926-130#00-XYz_DDK-130_vX.Y[.Z].tar.gz
# cd DDK-130_vX.Y[.Z]
# ./install_ddk.sh
{INSTALL] Please check for availability of recent DDK patches!

[INSTALL] Do you want to set the DDK environment variables [y/n]?
y
...
[INSTALL] Do you want to build the DDK components now [y/n]?
y
...
[INSTALL] Installation completed successfully
#
```

This script will first ask whether the *DDK* packages must be built. If the question is answered with “y”, then this script will prompt for the environment variables required to build the *DDK* and will build the Example Drivers, VAL Test Tool and Demo Applications. Otherwise, the script will just unpack and install all the software packages included in the *DDK*. This table shows folders that are available after installation:

Table 2 *DDK* folders after installation

Folder name	Folder content
Config	The cs_*.h files that hold the generic configuration settings.
docs	All DDK related documents: • DDK User Guide, SW Architecture, Test Specification and Test Report • Driver User Guide and SW Block Diagram • Driver Library User Guide(s) • Driver Framework Porting Guide • Release Notes
Examples/Driver	*EIP-130* Example Combined Driver for the *EIP-130* hardware.
Examples/Driver_GC	*EIP-130* Example Global Control Driver for the *EIP-130* hardware.
Examples/Driver_KernelSupport	Kernel Support Driver including the User-Space device access (UMDevXS) functionality.
Examples/Driver_VAL	*EIP-130* Example VAL Driver for the *EIP-130* hardware.
Examples/DemoApp_VAL	VAL Demo Application that demonstrates the VAL API and its use.
Examples/DemoApp_VAL_Specials	Demo Application that uses special functions of the VAL API that can make the *EIP-130* hardware unusable.
Examples/DemoApp_SecureDebug	Demo Application to demonstrate the Authenticated Unlock and Secure Debug functionality.
Examples/DemoApp_OTPProgram	Demo Application with which an OTP item can be written / programmed. Note that the OTP item must be provided as key blob to ensure the item authentication and integrity.
Examples/DemoApp_OTPKeyBlob	Demo Application that uses the *EIP-130* hardware to generate a key blob of an OTP item. This generated OTP item key blob could be written to OTP with DemoApp_OTPProgram.

Folder name	Folder content
TestTools/MemXS	Diagnostics tool that can be used to read and write the *EIP-130* device registers. This tool can also be used to verify the correctness of the Driver Framework Device API implementation for User-Space deployment (located in Kit/DriverFramework/src/UMDevXS).
TestTools/Regress_vip	Regression test system that runs the VAL Demo Application and VAL Test Tool on the *EIP-130* driver for a set of pre-defined configurations. The provided regression scripts automate the build process and the regression.
TestTools/TestTool_VAL	VAL Test Tool source code for the VAL API.
Kit/DriverFramework	The Driver Framework APIs Specification as C header-files.
Kit/EIP130	The *EIP-130* Driver Library Specification as C header-files and the implementation for Global Control and Mailbox Control functions. Additionally, the library contains the TokenHelper functions with which tokens can be created and parsed.
Kit/EIP201	The *Security IP-201* Driver Library implementation that can be used in drivers to handle interrupt functionality.
Kit/Log	Logging API Specification as C header-files and example implementation of the API as used in the *EIP-130* driver.
Integration/Adapter_DriverInit	Driver initialization functions.
Integration/Adapter_Generic	Generic adapter functions.
Integration/Adapter_GlobalControl	Global Control specific functions.
Integration/Adapter_Interrupts	Implementation of interrupt handler registration.
Integration/Adapter_VAL	The VAL API implementation. The functions from the VAL API are converted to logical tokens that are passed on to the VEX layer.
Integration/Adapter_VEX	The VEX API implementation. The logical tokens from the VAL layer are converted to physical tokens, which are processed by the *EIP-130* hardware.
Integration/DriverFramework/src	Generic Driver Framework implementation code (common to all implementations).
Integration/DriverFramework/src/LKM	The Device API and the DMA Resource API implementation for Linux Kernel-Space x86 and RISC HW platforms. Depending on the package, the Device API implementation can be specialized for Open Firmware or PCI.
Integration/DriverFramework/src/UMDevXS	The Device API and the DMA Resource API implementation for User Mode device access (UMDevXS).
Integration/UMDevXS	The UMDevXS kernel and user part implementation of the Linux Kernel Support Driver and its User-Space module that provides the required kernel services to the *EIP-130* User-Space drivers. This Kernel Support Driver functionality is mainly used for the Linux User-Space driver deployment. This part of the driver also provides support for the UMDevXS part of the Kernel Support Driver and centralized device registration for all other drivers, except the Combined Driver.
Integration/VEX_Proxy_Stub	The VEX Proxy or Stub implementation that provides the required kernel services to the User-Space applications that use the VAL API. In the general, the VEX Proxy provides the communication between the application and *EIP-130* VAL or Combined Driver.

(End of Document)